choreographerExecutor

Service Description

Service ID: *"choreographerExecutor"*

Abstract
This document describes the `choreographerExecutor` service, version 1.0.0, produced by the Choreographer support system of Arrowhead (documentation edition 5.3.0).

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**
2.1 Significant Prior Art . . . 3
2.2 How This Service Is Meant to Be Used . . . 3
2.3 Important Delimitations and Dependencies . . . 3

3 Service Interface **4**
3.1 operation register . . . 4
3.2 operation unregister . . . 4
3.3 operation notify-step-done . . . 4

4 Information Model **5**
4.1 struct ExecutedStepResult . . . 5
4.2 struct ExecutorRegistrationRequest . . . 5
4.3 struct ExecutorResponse . . . 5
4.4 struct ExecutorServiceDefinitionResponse . . . 5
4.5 Primitives . . . 6

5 References **7**

6 Revision History **8**
6.1 Amendments . . . 8
6.2 Quality Assurance . . . 8

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the `choreographerExecutor` service, which provides registration of executor systems and reporting of executed plan steps. It is produced by the Choreographer system [1] in service version 1.0.0.

The rest of this document is organized as follows. In Section 3, we describe the abstract message operations provided by the service. In Section 4, we end the document by presenting the data types used by the mentioned operations. The concrete interface is specified in the HTTP/TLS/JSON IDD [2].

2.1 Significant Prior Art

Successor of the Arrowhead 4.6 Choreographer executor services (`notifyStepDone` became `notify-step-done`).

2.2 How This Service Is Meant to Be Used

An executor registers itself with `register` (address, port, base path and the service definition it executes steps for), reports each finished step with `notify-step-done`, and leaves with `unregister`.

2.3 Important Delimitations and Dependencies

The executor address must be one of its ServiceRegistry addresses; a step result is accepted only from the step's executor.

3 Service Interface

This section describes the interfaces to the `choreographerExecutor` service. In particular, each subsection names an abstract operation, an input type and an output type, in that order. The input type is named inside parentheses, while the output type is preceded by a colon. Input and output types are only denoted when accepted or returned, respectively, by the interface in question.

All abstract data types named in this section are defined in Section 4.

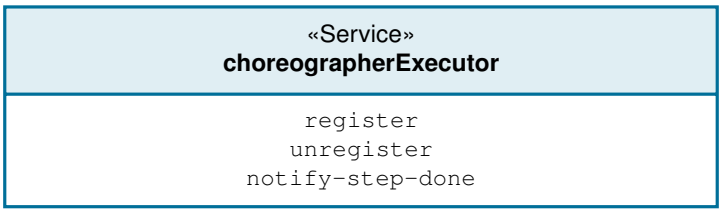


Figure 1: SysML-style block of the `choreographerExecutor` service and its operations.

The following interface operations are available.

3.1 operation register (ExecutorRegistrationRequest) : ExecutorResponse

Registers the requester as executor for a service definition (version window).

3.2 operation unregister ()

Removes the requester's executor registration.

3.3 operation notify-step-done (ExecutedStepResult)

Reports the result of a session step executed by the requester.

4 Information Model

Here, all data objects that can be part of the `choreographerExecutor` service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is used to denote a collection of named fields, each with its own data type. As a complement to the explicitly defined types in this section, there is also a list of implicit primitive types in Section 4.5, which are used to represent things like hashes and identifiers.

4.1 struct ExecutedStepResult

Result of an executed step, reported by the executor.

Field	Type	Description
sessionId	Number	Identifier of the session.
sessionStepId	Number	Identifier of the executed session step.
status	ExecutedStepStatus	Result: `SUCCESS`, `ERROR`, `FATAL_ERROR` or `ABORTED`.
message	String	Result message.
exception	String	Error details, if any.

4.2 struct ExecutorRegistrationRequest

Registration of the requesting executor.

Field	Type	Description
address	String	Address of the executor; must be one of its ServiceRegistry addresses.
port	Number	Port of the executor interface.
basePath	String	Base path of the executor interface.
serviceDefinitionName	Name	Service definition the executor can execute steps for.
minVersion	Number	Lowest accepted major version.
maxVersion	Number	Highest accepted major version.

4.3 struct ExecutorResponse

A registered executor.

Field	Type	Description
id	Number	Identifier of the executor.
name	Name	Executor system.
address	String	Address of the executor interface.
port	Number	Port of the executor interface.
basePath	String	Base path of the executor interface.
serviceDefinitions	List<ExecutorServiceDefinitionResponse>	Service definitions the executor serves.
createdAt	DateTime	UTC time of creation.
updatedAt	DateTime	UTC time of the last change.

4.4 struct ExecutorServiceDefinitionResponse

A service definition an executor serves.

Field	Type	Description
serviceDefinitionName	Name	Service definition name, camelCase (e.g. `temperatureReading`).
minVersion	Number	Lowest accepted major version.
maxVersion	Number	Highest accepted major version.

4.5 Primitives

Types and structures mentioned throughout this document that are assumed to be available to implementations of this service. The concrete interpretations of each of these types and structures must be provided by any IDD document claiming to implement this service.

Type	Description
DateTime	Pinpoints a specific moment in time (UTC).
ExecutedStepStatus	One of `SUCCESS`, `ERROR`, `FATAL_ERROR`, `ABORTED`.
List	An *array* of a known number of items, each having type A.
Name	A string identifier that is intended to be both human and machine-readable.
Number	An integer number.
String	A chain of characters.

5 References

[1] J. Delsing, “Choreographer – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, ah5-support-java-spring/Choreography/documentation/v5.3.0/Choreographer-v5.3.0-SysD.pdf.

[2] ——, “choreographerExecutor HTTP/TLS/JSON – Interface Design Description (Choreographer),” Eclipse Arrowhead, IDD 5.3.0, 2026, choreographer-v5.3.0-choreographerExecutor-HTTP-IDD.pdf.

6 Revision History

6.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged.	Claude Opus 5.5 (draft)
3	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

6.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	